PolyVote: Database-Enforced Integrity and a Keyed Audit Ledger for Small Ranked-Choice Elections

Tharakesharasu Thennarasu, M Pavan Durga Karthik and Shivam Malge
School of Computer Science and Engineering, Course BACSE202 Database Systems
Faculty guide: Dr. Katari Balakrishna

Abstract—Small elections are usually run on a general-purpose database that an administrator can edit without leaving a trace. Cryptographic end-to-end verifiable systems [1, 2] address this but carry operational burdens that campus and club elections rarely accept. We present PolyVote, a multi-tenant ranked-choice (instant-runoff) election platform that raises the cost of undetected tampering using database features alone. Constraints and a trigger enforce one vote per voter; voters and anonymous ballots are stored without a linking column; each ballot is sealed in an HMAC-keyed, hash-chained ledger kept in MongoDB Atlas; and an integrity sweep reconciles ballots, receipts, voter flags and the sealed candidate roster. A standalone email one-time-code module adds a second authentication factor. We evaluate the system with an executable suite of 47 scenarios. Against the earlier design, rogue-administrator scenarios detected automatically rise from 6 to 12 of 13, including forged receipts and consistent chain rewrites that a plain hash chain cannot stop; sweep time rises by about 14% at 10,000 ballots. One scenario, roster injection, remains undetected, and we state the limits of a design that is not end-to-end verifiable.

Index Terms—electronic voting, instant-runoff voting, tamper-evident logging, HMAC, database integrity, polyglot persistence, one-time password.

I. INTRODUCTION

Student councils, clubs and societies run elections on spreadsheets, survey forms or small web applications backed by one database. Whoever administers that database can change the outcome, and nothing in the system can show that a result was altered. Research on electronic voting has focused on cryptographic protocols that avoid trusting the server [1, 2, 3, 4, 5] and on the failures of deployed internet voting [6, 7, 8, 9]. Between the two lies a large population of small elections for which verifiable cryptographic voting is impractical, yet where much more can be done by using the database properly.

PolyVote treats the database as the place where election rules are enforced and adds an independent append-only audit trail that can be checked against it. Its contributions are: (1) a database-centric design that enforces one-vote and ballot-validity rules in the engine and separates voters from ballots; (2) an HMAC-keyed hash-chained ledger in a document store with a sealed candidate roster; (3) an extended integrity sweep; (4) a standalone email one-time-code module; and (5) a reproducible evaluation of 47 scenarios against both the earlier design and the final one, with an explicit statement of what is not claimed.

II. RELATED WORK

Helios [1] and Belenios [2] provide verifiable tallies using encrypted ballots; Civitas [3] adds coercion resistance; Prêt à Voter [4] and Scantegrity II [5] target paper-based verification. Analyses of deployed systems found significant weaknesses in Estonian internet voting [6] and in mobile and web products [7, 8], and a National Academies study concluded internet voting could not currently be secured [9]. Blockchain voting compounds these problems [10].

Hash chaining for time-stamping [11] and secure audit logs [12] underlie tamper-evident logging [13, 14]; database work covers tamper detection in audit logs [15], forensic analysis of tampering [16] and ledger tables in a commercial engine [17]. Instant-runoff outcomes can hinge on few ballots, motivating margin computation and risk-limiting audits [18, 19]. Multi-tenant schema mapping [20], polyglot persistence [21] and database security [22] complete the background. For authentication, NIST SP 800-63B separates memorised secrets from possession factors [23]; email [24] and mobile channels [25] are convenient but only as strong as the mailbox or device.

III. THREAT MODEL

We consider seven adversaries: a voter with a leaked password (A1); a network observer (A2, out of scope beyond TLS); a rogue administrator of the election database (A3); an administrator of the ledger store (A4); an insider with both stores but no secrets (A5); an insider who also holds the key (A6); and roster injection through the administrative path (A7). Goals are: no double vote; only rostered, verified voters vote; ballots anonymous in storage; malformed ballots rejected; modification, deletion or insertion detectable; the ledger not silently rewritable without the key; candidate attribution fixed after voting; and voters able to confirm their ballot is recorded. A6 and A7 are not covered.

ID	Adversary	Capability	PolyVote response	Status
A1	Voter with a leaked or guessed password	Signs in as someone else	Emailed one-time code; one-vote trigger	Prevented
A2	Network observer	Reads or alters traffic	Out of scope beyond TLS at the tunnel	Not covered
A3	Rogue database administrator	Read/write on the election database only	Integrity sweep: modified, deleted, injected, mismatch	Detected
A4	Ledger-store administrator	Read/write on the ledger store only	HMAC hash chain; unique block index	Detected
A5	Insider with both stores, no secrets	Edits both stores to match each other	Keyed chain + sweep catch forged receipts, rewrites	Detected
A6	Insider with both stores and the ledger key	Forges valid blocks	Needs external anchoring (future work)	Not covered
A7	Roster injection through the admin path	Adds a ghost voter, votes as them through the API	Needs a committed roster (future work)	Not covered

Fig. 1. Threat model: adversaries, capabilities and how each is treated.

Assumptions

The application host and its environment secrets (database connection string, ledger key, SMTP credentials) are not controlled by the adversary except in A6. A voter controls their mailbox. The election officer registers the roster honestly. Coercion, vote selling, denial of service and compromise of the voter's own device are out of scope.

IV. DESIGN

PolyVote is one FastAPI service with a React front end. It uses three stores separated by what each is trusted with: a relational election database, a separate verification database for emails and one-time codes, and a MongoDB Atlas ledger, with application secrets held outside all of them (Fig. 2). An administrator of one store cannot touch the others, and only the application host sees all three, which is why host compromise is outside the model.

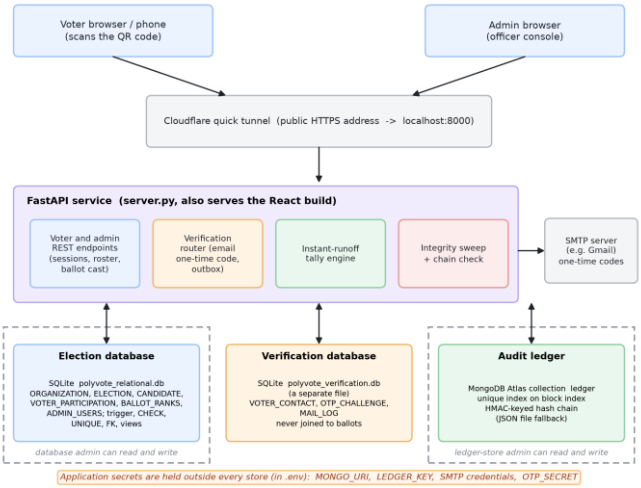


Fig. 2. Architecture: election database, separate verification database and MongoDB ledger, with secrets outside all stores.

A. Data model and rules

One shared schema serves many organisations [20]. Six tables hold organisations, elections, candidates, the roster and ranked ballot rows. The engine enforces: a trigger that aborts a second `has_voted` update, CHECK constraints on flags and rank (1–3), a composite key that prevents ranking a candidate twice, per-election uniqueness of registration numbers, and foreign keys [26, 27]. A ballot is stored under a random token; no column links it to a voter, and voter email addresses live in a different database (Fig. 3). An automated check confirms the two ballot and voter tables share no column other than the election identifier.

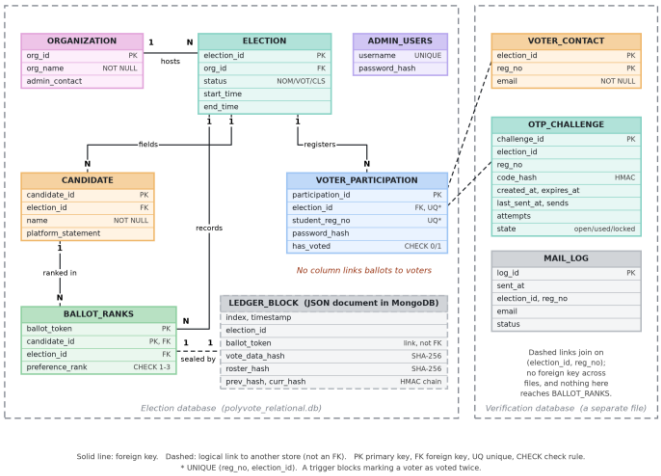


Fig. 3. Entity-relationship diagram: election database, verification database and ledger block.

B. Casting a ballot

Casting is serialised and transactional. After authentication the server takes a write lock, re-checks the election status, validates the ranking and confirms the voter has not voted. In one transaction it sets the voted flag and inserts the ranked rows under a new random token, then appends a sealed block to the ledger and invalidates the session. The append follows the commit, so a failure between them leaves a ballot with no receipt, which the sweep reports as injected; we chose the variant that fails visibly. Tallying is instant-runoff: first preferences are counted among continuing candidates, a candidate with more than half wins, otherwise the weakest is eliminated and ballots transfer. Ties for last place are broken by candidate order, and the interface flags such rounds.

C. Keyed audit ledger

Each ballot yields one block holding its index, election, token, a SHA-256 of the ranked choices, a SHA-256 of the candidate roster, the previous hash and its own hash. The block hash is HMAC-SHA-256 under a secret held outside both stores, so an attacker who can write the ledger but lacks the key cannot rebuild a consistent chain, which a plain hash chain permits. In MongoDB a unique index on the block position prevents two writers from forking the chain; a local file is the fallback.

D. Integrity sweep

The sweep applies five rules: modified ballots (recomputed hash differs from the receipt), deleted ballots (receipt without rows), injected ballots (rows without receipt), changed roster (candidate names differ from the sealed hash), and count mismatch among voter flags, distinct ballots and receipts. A separate chain check verifies every link and block hash.

E. Email one-time code

After a correct password the server issues a challenge rather than a session. A six-digit code from a cryptographic generator is stored only as an HMAC, emailed from a background thread, valid for five minutes, single use, limited to three guesses, with resends limited to one per 30 s and five

per challenge, and at most five challenges per voter per 15 minutes. Failure of mail delivery never blocks sign-in; the administrator outbox shows issued codes (Fig. 4). Voter addresses live in a database separate from ballots, so an email address cannot be joined to a ballot even if the election database is compromised.

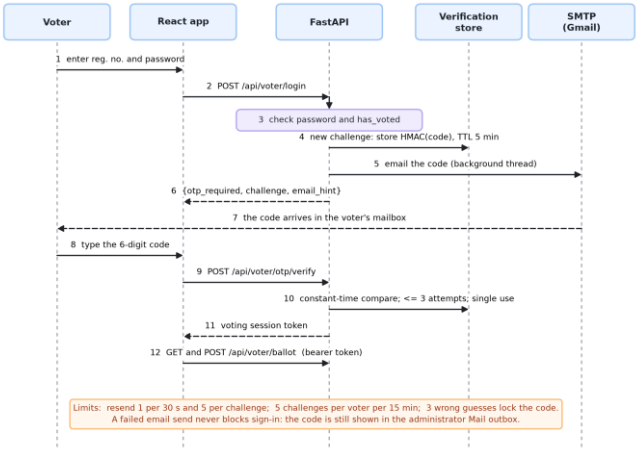


Fig. 4. Sign-in with a one-time code.

V. EVALUATION

An executable suite copies the application to a temporary directory, drives the real API, and plays a rogue administrator by editing the database and ledger files directly. Groups are: 7 database-level prevention tests, 13 API abuse tests, 13 tampering scenarios, 14 OTP tests, and benchmarks at 100, 1,000 and 10,000 ballots (median of three runs). The baseline is the earlier design, checked out and run unchanged; attacker scripts never receive the key.

A. Prevention and abuse

Six of seven database-level statements are rejected by the engine. The exception is a dangling candidate reference inserted from a console with default settings, because SQLite enforces foreign keys per connection. All 13 API abuse tests are refused, including twenty simultaneous casts by one voter, of which exactly one is stored. All 14 OTP tests pass; test F4 initially failed and exposed a defect in which failed guesses were rolled back with the aborted request, leaving the attempt limit ineffective.

B. Tampering

TABLE I. Outcome of each tampering scenario.

ID	Scenario	Earlier	Final
C1	Swap the 1st and 2nd choice on one ballot	Detected	Detected
C2	Replace the first-choice candidate on one ballot	Detected	Detected
C3	Delete one ballot	Detected	Detected
C4	Delete a ballot and clear a voted flag to hide the gap	Detected	Detected
C5	Inject a ballot with no receipt	Counters only	Detected
C6	Ballot stuffing: inject ballot, forge receipt, set a voted flag	Not detected	Detected
C7	Edit a ledger block without repairing the chain	Detected	Detected
C8	Delete a block from the middle of the ledger	Detected	Detected
C9	Truncate the last block of the ledger	Counters only	Detected
C10	Change a ballot and rebuild the whole chain consistently (attacker has no secret key)	Not detected	Detected
C11	Swap two candidates' names after voting	Not detected	Detected
C12	Reset a voted flag, then let that voter cast a second ballot	Counters only	Detected
C13	Add a ghost voter to the roster and vote as them through the API	Not detected	Not detected

Under the earlier design 6 of 13 scenarios are detected automatically, 3 only through counters an officer would have to compare by eye, and 4 not at all. The final design detects 12 and misses one. Forging a receipt (C6) and rebuilding the whole chain (C10) succeed against a plain chain and fail against the keyed one. Renaming candidates (C11) is caught by the sealed roster. C13, adding a ghost voter and voting as them, is undetected: the ballot is valid, and the system holds no independent record of the roster.

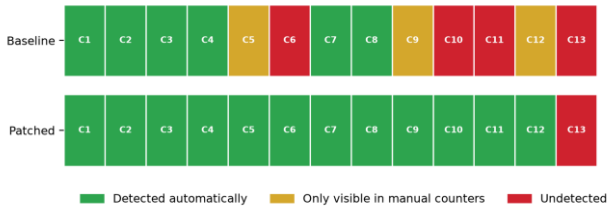


Fig. 5. Detection outcome per scenario, earlier design versus final design.

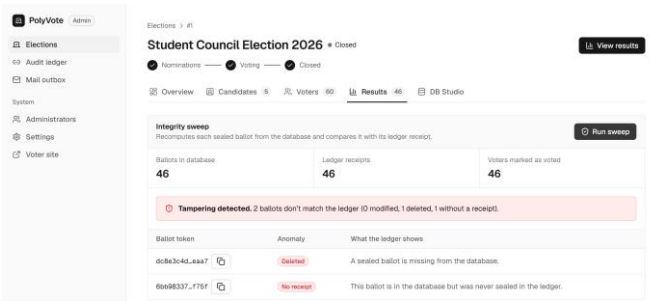


Fig. 6. Sweep after one ballot was deleted and another injected: both are reported although the three counters still read 46/46/46.

Fig. 6 matters because it shows why counters alone are not enough: a deletion and an injection cancel out in the counts, yet the sweep names both ballots.

C. One-time-code tests

The 14 OTP tests check that a correct password alone yields no session; that the right code opens the ballot; that a used code cannot be replayed; that three wrong codes lock the challenge; that expired codes, rapid resends and a sixth code are refused; that a voter without an email on file cannot proceed; that plaintext codes are absent from the verification database; that repeated sign-ins are rate-limited; that codes are six digits with leading zeros preserved; that an unreachable mail server does not block sign-in; and that the election database has no email column.

D. Performance

TABLE II. File-ledger timings, earlier / final design.

Ballots	Cast ms	Sweep ms	Chain ms	Tally ms
100	9 / 10	2 / 4	1 / 2	8 / 12
1,000	33 / 29	17 / 12	7 / 10	11 / 9

10,000	205 / 207	129 / 147	80 / 102	57 / 60

Read-side operations grow linearly. The keyed chain and extra rules cost about 15% on the sweep at 10,000 ballots. With the file ledger a cast grows from 9 ms to 205 ms, since each append rewrites the file. Against MongoDB Atlas over a mobile hotspot an append costs about 147 ms regardless of chain length; the file is faster at small sizes and the crossover is near ten thousand ballots. Verifying the whole chain over the network grows from 0.2 s at 100 blocks to 3.0 s at 10,000, so a production system should verify incrementally from a trusted checkpoint.

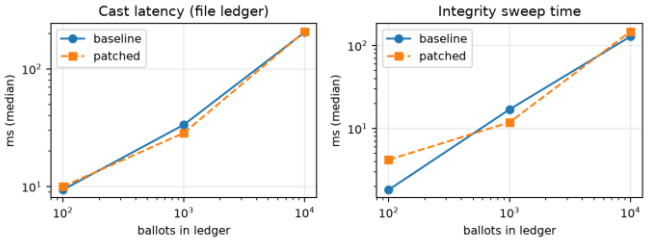


Fig. 7. Cast latency and sweep time against ledger size (log scales).

E. Threats to validity

The workload is synthetic and timings come from one laptop and one mobile network, so we report trends rather than absolute values. The attack scenarios are the ones we thought of and are expressed against our own design, so they are not a complete catalogue. The baseline is our earlier version; we did not run Helios or Belenios, so Section VI is qualitative. Detection also depends on the officer running the checks.

VI. COMPARISON

TABLE III. Qualitative comparison (published designs [1, 2, 3, 4, 5, 6, 7, 8]).

System	Assurance	Voter verifies	Burden
Helios, Belenios	Encrypted, verifiable tally	Yes	Moderate
Civitas, Prêt à Voter, Scantegrity II	Verifiable, some coercion-resistant	Yes	High
Estonian i-voting	National crypto system	Partial	High
Commercial mobile/web	Vendor trust	No	Low
PolyVote	Database rules; keyed ledger	Recorded, not content	Low

TABLE IV. Earlier design versus final design.

Measure	Earlier design	Final design
Tampering detected automatically	6 of 13	12 of 13
Forged receipt / chain rewrite	Undetected	Detected (key)
Candidate renaming	Undetected	Detected (roster seal)
DB prevention / API abuse tests	6/7, 13/13	6/7, 13/13
Voter authentication	Password	Password + emailed code
Ledger storage	Local file	MongoDB Atlas
Sweep at 10,000 ballots	129 ms	147 ms

PolyVote sits at the low-assurance, low-burden end and is not a substitute for verifiable systems where a hostile administrator is the main concern. We claim an improvement over the earlier design and over unprotected relational election applications, not over end-to-end verifiable systems.

VII. LIMITATIONS AND FUTURE WORK

- An insider holding the key and both stores can forge a consistent chain (A6). Publishing the chain head to an external witness, as transparency logs do [14], would address this.
- Roster injection (A7) is not detected; a committed or signed roster is needed.
- Receipts prove a ballot is recorded and counted, not its content [28]; coercion is not resisted [29].
- Passwords use plain SHA-256; salted, slow hashing is recommended [30, 31]. Email is a weak factor [24]; TOTP [32] is a stronger alternative.
- Foreign-key enforcement depends on the connection; a server database removes this. Oracle with PL/SQL, roles and an entity for election positions were deferred.

VIII. CONCLUSION

Using database features alone, PolyVote enforces one-person-one-vote in the engine, stores voters and ballots without a linking column, seals each ballot in a keyed ledger and exposes tampering through a reconciling sweep, with a second authentication factor for sign-in. Of 13 tampering scenarios, the earlier design detected 6 automatically and the final design detects 12, at a small run-time cost. We report one undetected scenario and the limits of the approach, and position the system as a practical option for small elections rather than a rival to cryptographic voting.

REFERENCES

[1] B. Adida, “Helios: Web-based Open-Audit Voting,” in Proc. 17th USENIX Security Symposium (USENIX Security 08), San Jose, CA, 2008. [Online]. Available: https://www.usenix.org/conference/17th-usenix-security-symposium/helios-web-based-open-audit-voting

[2] V. Cortier, P. Gaudry, and S. Glondu, “Belenios: A Simple Private and Verifiable Electronic Voting System,” in Foundations of Security, Protocols, and Equational Reasoning (Lecture Notes in Computer Science), Springer, 2019, pp. 214–238. doi: 10.1007/978-3-030-19052-1_14.

[3] M. R. Clarkson, S. Chong, and A. C. Myers, “Civitas: Toward a Secure Voting System,” in Proc. 2008 IEEE Symposium on Security and Privacy, 2008, pp. 354–368. doi: 10.1109/sp.2008.32.

[4] P. Y. A. Ryan, D. Bismark, J. Heather, S. Schneider, and Z. Xia, “Prêt à Voter: a Voter-Verifiable Voting System,” IEEE Transactions on Information Forensics and Security, vol. 4, no. 4, pp. 662–673, 2009. doi: 10.1109/TIFS.2009.2033233.

[5] D. Chaum et al., “Scantegrity II: End-to-End Verifiability by Voters of Optical Scan Elections Through Confirmation Codes,” IEEE Transactions on Information Forensics and Security, vol. 4, no. 4, pp. 611–627, 2009. doi: 10.1109/tifs.2009.2034919.

[6] D. Springall et al., “Security Analysis of the Estonian Internet Voting System,” in Proc. 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS '14), 2014, pp. 703–715. doi: 10.1145/2660267.2660315.

[7] M. A. Specter, J. Koppel, and D. Weitzner, “The Ballot is Busted Before the Blockchain: A Security Analysis of Voatz, the First Internet Voting Application Used in U.S. Federal Elections,” in Proc. 29th USENIX Security Symposium (USENIX Security 20), 2020, pp. 1535–1553. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/specter

[8] M. A. Specter and J. A. Halderman, “Security Analysis of the Democracy Live Online Voting System,” in Proc. 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 3077–3092. [Online]. Available:

https://www.usenix.org/conference/usenixsecurity21/presentation/specter-security

[9] National Academies of Sciences, Engineering, and Medicine, Securing the Vote: Protecting American Democracy. National Academies Press, Washington, DC, 2018. doi: 10.17226/25120.

[10] S. Park, M. Specter, N. Narula, and R. L. Rivest, "Going from bad to worse: from Internet voting to blockchain voting," Journal of Cybersecurity, vol. 7, no. 1, 2021. doi: 10.1093/cybsec/tyaa025.

[11] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," Journal of Cryptology, vol. 3, no. 2, pp. 99–111, 1991. doi: 10.1007/BF00196791.

[12] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," ACM Transactions on Information and System Security, vol. 2, no. 2, pp. 159–176, 1999. doi: 10.1145/317087.317089.

[13] S. A. Crosby and D. S. Wallach, "Efficient Data Structures for Tamper-Evident Logging," in Proc. 18th USENIX Security Symposium (USENIX Security 09), Montreal, Canada, 2009. [Online]. Available: https://www.usenix.org/conference/usenixsecurity09/technical-sessions/presentation/efficient-data-structures-tamper-evident

[14] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," IETF, RFC 6962, 2013 (Experimental). [Online]. Available: https://www.rfc-editor.org/info/rfc6962

[15] R. T. Snodgrass, S. S. Yao, and C. Collberg, "Tamper Detection in Audit Logs," in Proc. 2004 VLDB Conference, 2004, pp. 504–515. doi: 10.1016/b978-012088469-8.50046-2.

[16] K. E. Pavlou and R. T. Snodgrass, "Forensic analysis of database tampering," ACM Transactions on Database Systems, vol. 33, no. 4, pp. 1–47, 2008. doi: 10.1145/1412331.1412342.

[17] P. Antonopoulos et al., "SQL Ledger: Cryptographically Verifiable Data in Azure SQL Database," in Proc. 2021 International Conference on Management of Data (SIGMOD '21), 2021, pp. 2437–2449. doi: 10.1145/3448016.3457558.

[18] T. R. Magrino, R. L. Rivest, E. Shen, and D. Wagner, "Computing the Margin of Victory in IRV Elections," in Proc. 2011 Electronic Voting Technology Workshop / Workshop on Trustworthy Elections (EVT/WOTE '11), San Francisco, CA, 2011. [Online]. Available: https://www.usenix.org/conference/evtwote-11/computing-margin-victory-irv-elections

[19] M. Blom, P. J. Stuckey, and V. Teague, "RAIRE: Risk-Limiting Audits for IRV Elections," arXiv preprint arXiv:1903.08804 [cs.DS], 2019. [Online]. Available: https://arxiv.org/abs/1903.08804

[20] S. Aulbach, T. Grust, D. Jacobs, A. Kemper, and J. Rittinger, "Multi-tenant databases for software as a service: schema-mapping techniques," in Proc. 2008 ACM SIGMOD International Conference on Management of Data, 2008, pp. 1195–1206. doi: 10.1145/1376616.1376736.

[21] P. J. Sadalage and M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley Professional, 2012. [Online]. Available: https://martinfowler.com/books/nosql.html

[22] E. Bertino and R. Sandhu, "Database security - concepts, approaches, and challenges," IEEE Transactions on Dependable and Secure Computing, vol. 2, no. 1, pp. 2–19, 2005. doi: 10.1109/TDSC.2005.9.

[23] D. Temoshok et al., "Digital Identity Guidelines: Authentication and Authenticator Management," NIST Special Publication 800-63B-4, National Institute of Standards and Technology, 2025. doi: 10.6028/NIST.SP.800-63b-4.

[24] S. L. Garfinkel, "Email-based identification and authentication: an alternative to PKI?," IEEE Security & Privacy, vol. 1, no. 6, pp. 20–26, 2003. doi: 10.1109/MSECP.2003.1253564.

[25] A. Dmitrienko, C. Liebchen, C. Rossow, and A.-R. Sadeghi, "On the (In)Security of Mobile Two-Factor Authentication," in Financial Cryptography and Data Security (FC 2014), Lecture Notes in Computer Science, Springer, 2014, pp. 365–383. doi: 10.1007/978-3-662-45472-5_24.

[26] "PostgreSQL 18 Documentation, Section 5.5: Constraints," PostgreSQL Global Development Group, PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/current/ddl-constraints.html (accessed Oct. 22, 2026).

[27] "PostgreSQL 18 Documentation, Chapter 37: Triggers," PostgreSQL Global Development Group, PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/current/triggers.html (accessed Oct. 22, 2026).

[28] D. Chaum, "Secret-ballot receipts: true voter-verifiable elections," IEEE Security & Privacy, vol. 2, no. 1, pp. 38–47, 2004. doi: 10.1109/MSECP.2004.1264852.

[29] A. Juels, D. Catalano, and M. Jakobsson, "Coercion-resistant electronic elections," in Proc. 2005 ACM Workshop on Privacy in the Electronic Society (WPES '05), 2005, pp. 61–70. doi: 10.1145/1102199.1102213.

[30] M. Sönmez Turan, E. Barker, W. Burr, and L. Chen, "Recommendation for Password-Based Key Derivation: Part 1: Storage Applications," NIST Special Publication 800-132, National Institute of Standards and Technology, 2010. doi: 10.6028/NIST.SP.800-132.

[31] OWASP, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series, OWASP Foundation. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed Oct. 22, 2026).

[32] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," IETF, RFC 6238, 2011 (Informational). [Online]. Available: https://www.rfc-editor.org/info/rfc6238